Data Structures & Algorithms — Complete Interview Guide

Common Problems, Approach, Logic & Python Solutions — for SDE/CSE Roles

Study Notes

Contents

1 How to Use This Guide **3**

2 1. Arrays **4**
- 2.1 1.1 Concept . . . 4
- 2.2 1.2 Problem: Two Sum . . . 4
- 2.3 1.3 Problem: Kadane's Algorithm (Maximum Subarray Sum) . . . 4
- 2.4 1.4 Problem: Sliding Window Maximum / Longest Subarray with Sum ≤ K . . . 4
- 2.5 1.5 Problem: Move Zeroes to End (In-place) . . . 5
- 2.6 1.6 Problem: Trapping Rain Water . . . 5
- 2.7 1.7 Problem: Kth Largest Element . . . 6
- 2.8 1.8 Practice List (Solve Independently) . . . 6

3 2. Strings **7**
- 3.1 2.1 Concept . . . 7
- 3.2 2.2 Problem: Longest Substring Without Repeating Characters . . . 7
- 3.3 2.3 Problem: Valid Anagram . . . 7
- 3.4 2.4 Problem: Check if a String is a Palindrome (Two Pointer) . . . 7
- 3.5 2.5 Problem: Longest Palindromic Substring . . . 8
- 3.6 2.6 Problem: String Compression / Run-Length Encoding . . . 8
- 3.7 2.7 Practice List . . . 8

4 3. Linked Lists **10**
- 4.1 3.1 Concept . . . 10
- 4.2 3.2 Problem: Reverse a Linked List . . . 10
- 4.3 3.3 Problem: Detect Cycle in a Linked List (Floyd's Cycle Detection) . . . 10
- 4.4 3.4 Problem: Find the Middle of a Linked List . . . 10
- 4.5 3.5 Problem: Merge Two Sorted Linked Lists . . . 11
- 4.6 3.6 Problem: Detect and Remove a Cycle's Starting Point . . . 11
- 4.7 3.7 Practice List . . . 11

5 4. Stacks and Queues **13**
- 5.1 4.1 Concept . . . 13
- 5.2 4.2 Problem: Valid Parentheses . . . 13
- 5.3 4.3 Problem: Next Greater Element . . . 13
- 5.4 4.4 Problem: Implement a Queue Using Two Stacks . . . 14
- 5.5 4.5 Problem: Sliding Window Maximum . . . 14
- 5.6 4.6 Practice List . . . 15

6 5. Trees **16**
6.1 5.1 Concept . . . 16
6.2 5.2 Traversals . . . 16
6.3 5.3 Problem: Maximum Depth of Binary Tree . . . 17
6.4 5.4 Problem: Validate a Binary Search Tree . . . 17
6.5 5.5 Problem: Lowest Common Ancestor (LCA) in a Binary Tree . . . 17
6.6 5.6 Problem: Diameter of a Binary Tree . . . 18
6.7 5.7 Problem: Serialize and Deserialize a Binary Tree . . . 18
6.8 5.8 Practice List . . . 19

7 6. Graphs **20**
7.1 6.1 Concept . . . 20
7.2 6.2 Problem: Depth-First Search (DFS) and Breadth-First Search (BFS) . . . 20
7.3 6.3 Problem: Detect Cycle in a Graph . . . 20
7.4 6.4 Problem: Topological Sort . . . 21
7.5 6.5 Problem: Number of Islands . . . 22
7.6 6.6 Problem: Dijkstra's Algorithm (Shortest Path, Weighted, Non-negative) . . . 22
7.7 6.7 Problem: Union-Find (Disjoint Set Union) — for Connected Components / Cycle Detection . . . 23
7.8 6.8 Practice List . . . 23

8 7. Dynamic Programming (DP) **24**
8.1 7.1 Concept . . . 24
8.2 7.2 Problem: Fibonacci Number (the "hello world" of DP) . . . 24
8.3 7.3 Problem: 0/1 Knapsack . . . 24
8.4 7.4 Problem: Longest Common Subsequence (LCS) . . . 25
8.5 7.5 Problem: Longest Increasing Subsequence (LIS) . . . 25
8.6 7.6 Problem: Coin Change (Minimum Coins) . . . 26
8.7 7.7 Problem: Edit Distance (Levenshtein Distance) . . . 26
8.8 7.8 Problem: House Robber . . . 27
8.9 7.9 Practice List . . . 27

9 8. Greedy Algorithms **28**
9.1 8.1 Concept . . . 28
9.2 8.2 Problem: Activity Selection / Non-overlapping Intervals . . . 28
9.3 8.3 Problem: Fractional Knapsack . . . 28
9.4 8.4 Problem: Jump Game (Minimum Jumps / Reachability) . . . 29
9.5 8.5 Practice List . . . 29

10 Complexity Cheat Sheet **30**

1 How to Use This Guide

For each topic:

1. **Concept** — what it is, when to use it, complexity.
2. **Common Problems** — the patterns that show up repeatedly in interviews/tests.
3. Each problem includes: **Approach & Logic**, then a clean **Python solution** with complexity noted.

Solve each problem yourself first before reading the solution — that's how the pattern actually sticks.

2 1. Arrays

2.1 1.1 Concept

Arrays store elements in contiguous memory, giving O(1) random access but O(n) insertion/deletion (due to shifting). Most interview array problems reduce to a handful of reusable patterns: **two pointers, sliding window, prefix sums, and sorting-based tricks**.

2.2 1.2 Problem: Two Sum

Statement: Given an array and a target, find indices of two numbers that add up to the target.

Approach: Brute force is O(n^2) checking every pair. Better: use a hash map to store `value → index` as you iterate; for each element, check if `target - element` already exists in the map. This gives O(n) time, O(n) space.

```python
def two_sum(nums, target):
    seen = {}  # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# Example
print(two_sum([2, 7, 11, 15], 9))  # [0, 1]
```

Complexity: Time O(n), Space O(n).

2.3 1.3 Problem: Kadane's Algorithm (Maximum Subarray Sum)

Statement: Find the contiguous subarray with the largest sum.

Approach: Track `current_sum` — at each element, decide whether to extend the previous subarray or start fresh from the current element (whichever gives a larger sum). Track the maximum seen so far. This is a classic Dynamic Programming pattern in disguise.

```python
def max_subarray(nums):
    current_sum = max_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))  # 6 ([4,-1,2,1])
```

Complexity: Time O(n), Space O(1).

2.4 1.4 Problem: Sliding Window Maximum / Longest Subarray with Sum ≤ K

Statement: Find the length of the longest subarray whose sum is ≤ K (all positive numbers).

Approach: Use a variable-size sliding window with two pointers `left` and `right`. Expand `right` to add elements; when the window sum exceeds K, shrink from `left`. Track the max window length seen.

```python
def longest_subarray_sum_k(nums, k):
    left = 0
    window_sum = 0
    max_len = 0
    for right in range(len(nums)):
        window_sum += nums[right]
        while window_sum > k and left <= right:
            window_sum -= nums[left]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len

print(longest_subarray_sum_k([1, 2, 1, 0, 1, 1, 0], 4))  # 5
```

Complexity: Time O(n) — each pointer moves forward at most n times, Space O(1).

2.5 1.5 Problem: Move Zeroes to End (In-place)

Statement: Move all zeroes in an array to the end while keeping the relative order of non-zero elements, in-place.

Approach: Use a "write pointer" — iterate with a read pointer; whenever a non-zero element is found, place it at the write pointer's position and increment write pointer. After the pass, fill the remaining positions with zeros.

```python
def move_zeroes(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
    return nums

print(move_zeroes([0, 1, 0, 3, 12]))  # [1, 3, 12, 0, 0]
```

Complexity: Time O(n), Space O(1).

2.6 1.6 Problem: Trapping Rain Water

Statement: Given an array representing elevation heights, compute how much water it can trap after raining.

Approach: Water trapped above index `i` = `min(max_left[i], max_right[i]) - height[i]`. Precompute max height to the left and right of every index (or use two pointers to avoid extra space).

```python
def trap_rain_water(height):
    if not height:
        return 0
    left, right = 0, len(height) - 1
    left_max, right_max = height[left], height[right]
    water = 0
```

```
    while left < right:
        if left_max < right_max:
            left += 1
            left_max = max(left_max, height[left])
            water += left_max - height[left]
        else:
            right -= 1
            right_max = max(right_max, height[right])
            water += right_max - height[right]
    return water

print(trap_rain_water([0,1,0,2,1,0,1,3,2,1,2,1]))  # 6
```

Complexity: Time O(n), Space O(1) (two-pointer version).

2.7 1.7 Problem: Kth Largest Element

Statement: Find the kth largest element in an unsorted array.

Approach: Sorting gives O(n log n). Better: use a **min-heap of size k** — push elements, and if heap size exceeds k, pop the smallest. The heap's root is the answer. (Quickselect gives average O(n) but is more complex to implement correctly.)

```
import heapq

def kth_largest(nums, k):
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]

print(kth_largest([3,2,1,5,6,4], 2))  # 5
```

Complexity: Time O(n log k), Space O(k).

2.8 1.8 Practice List (Solve Independently)

- Best Time to Buy and Sell Stock (single transaction)
- Product of Array Except Self (no division allowed)
- 3Sum (find all unique triplets summing to zero)
- Merge Intervals
- Find Missing Number in array of 1 to n
- Rotate Array by k positions

3 2. Strings

3.1 2.1 Concept

Strings are essentially arrays of characters with the same two-pointer/sliding-window patterns applying. Key extra tools: hash maps for character frequency, and recognizing palindrome patterns.

3.2 2.2 Problem: Longest Substring Without Repeating Characters

Approach: Sliding window with a set/hash map tracking characters currently in the window. Expand right; if a duplicate is found, shrink from the left until the duplicate is removed.

```
def longest_unique_substring(s):
    seen = {}
    left = 0
    max_len = 0
    for right, char in enumerate(s):
        if char in seen and seen[char] >= left:
            left = seen[char] + 1
        seen[char] = right
        max_len = max(max_len, right - left + 1)
    return max_len

print(longest_unique_substring("abcabcbb"))  # 3 ("abc")
```

Complexity: Time O(n), Space O(min(n, charset size)).

3.3 2.3 Problem: Valid Anagram

Approach: Two strings are anagrams if they have identical character frequency counts. Use a hash map (or fixed-size array for lowercase letters) to count and compare.

```
from collections import Counter

def is_anagram(s, t):
    return Counter(s) == Counter(t)

print(is_anagram("listen", "silent"))  # True
```

Complexity: Time O(n), Space O(1) for fixed alphabet.

3.4 2.4 Problem: Check if a String is a Palindrome (Two Pointer)

Approach: Use two pointers from both ends moving inward, comparing characters. Skip non-alphanumeric characters if required by the problem.

```
def is_palindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
```

```
    return True

print(is_palindrome("A man, a plan, a canal: Panama"))  # True
```

Complexity: Time O(n), Space O(n) for the filtered list.

3.5 2.5 Problem: Longest Palindromic Substring

Approach: "Expand around center" — every palindrome has a center (single char for odd length, between two chars for even length). Try expanding outward from each of the 2n-1 possible centers, tracking the longest found.

```
def longest_palindromic_substring(s):
    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1:right]

    result = ""
    for i in range(len(s)):
        odd = expand(i, i)
        even = expand(i, i + 1)
        result = max(result, odd, even, key=len)
    return result

print(longest_palindromic_substring("babad"))  # "bab" or "aba"
```

Complexity: Time O(n^2), Space O(1) excluding output.

3.6 2.6 Problem: String Compression / Run-Length Encoding

Approach: Iterate through the string, counting consecutive repeated characters, appending `char + count` to the result when the character changes.

```
def compress(s):
    result = []
    i = 0
    while i < len(s):
        char = s[i]
        count = 0
        while i < len(s) and s[i] == char:
            count += 1
            i += 1
        result.append(char + (str(count) if count > 1 else ""))
    return "".join(result)

print(compress("aaabbc"))  # "a3b2c"
```

Complexity: Time O(n), Space O(n).

3.7 2.7 Practice List

- Group Anagrams

- Minimum Window Substring
- String to Integer (atoi)
- Reverse Words in a String
- Implement strStr() (substring search / KMP algorithm)

4 3. Linked Lists

4.1 3.1 Concept

A linked list is a chain of nodes, each holding data and a pointer to the next node. Insert/delete at a known position is O(1) (no shifting), but access is O(n) (no random indexing). Master **fast-slow pointers** and **reversal via pointer manipulation** — they cover most linked-list interview questions.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

4.2 3.2 Problem: Reverse a Linked List

Approach: Maintain three pointers: `prev`, `curr`, `next_temp`. At each step, reverse the current node's pointer to point backward, then advance all three pointers forward.

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp
    return prev  # new head
```

Complexity: Time O(n), Space O(1).

4.3 3.3 Problem: Detect Cycle in a Linked List (Floyd's Cycle Detection)

Approach: Use two pointers, `slow` (moves 1 step) and `fast` (moves 2 steps). If there's a cycle, they will eventually meet inside it. If `fast` reaches `None`, there's no cycle.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Complexity: Time O(n), Space O(1).

4.4 3.4 Problem: Find the Middle of a Linked List

Approach: Same fast-slow pointer idea — when `fast` reaches the end, `slow` is at the middle (since it moves half as fast).

```
def find_middle(head):
    slow = fast = head
```

```
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Complexity: Time O(n), Space O(1).

4.5 3.5 Problem: Merge Two Sorted Linked Lists

Approach: Use a dummy head node to simplify edge cases. Compare the current nodes of both lists, attach the smaller one, and advance that list's pointer. Append any remaining nodes at the end.

```
def merge_two_lists(l1, l2):
    dummy = ListNode()
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

Complexity: Time O(n + m), Space O(1) (excluding output list itself).

4.6 3.6 Problem: Detect and Remove a Cycle's Starting Point

Approach: After detecting a cycle with fast-slow pointers, reset one pointer to head. Move both pointers one step at a time — they will meet exactly at the start of the cycle (a property derived from the distance math of Floyd's algorithm).

```
def detect_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            slow2 = head
            while slow2 != slow:
                slow2 = slow2.next
                slow = slow.next
            return slow2  # start of cycle
    return None
```

Complexity: Time O(n), Space O(1).

4.7 3.7 Practice List

- Remove Nth Node From End of List
- Add Two Numbers (represented as linked lists)

- Palindrome Linked List
- Flatten a Multilevel Doubly Linked List
- LRU Cache (uses doubly linked list + hash map)

5 4. Stacks and Queues

5.1 4.1 Concept

Stack = LIFO (Last In First Out); Queue = FIFO (First In First Out). Stacks are the backbone of parsing, backtracking, and "next greater/smaller element" problems. Queues (and deques) power BFS and sliding-window-max problems.

```
# Stack: use a Python list with append()/pop()
stack = []
stack.append(1)  # push
stack.pop()       # pop

# Queue: use collections.deque for O(1) operations at both ends
from collections import deque
queue = deque()
queue.append(1)      # enqueue at right
queue.popleft()       # dequeue from left
```

5.2 4.2 Problem: Valid Parentheses

Approach: Push opening brackets onto a stack. On a closing bracket, check if it matches the top of the stack (pop and compare); if not, or the stack is empty, it's invalid. At the end, the stack must be empty.

```
def is_valid_parentheses(s):
    stack = []
    pairs = {')': '(', ']': '[', '}': '{'}
    for char in s:
        if char in "([{":
            stack.append(char)
        else:
            if not stack or stack.pop() != pairs[char]:
                return False
    return not stack

print(is_valid_parentheses("()[]{}"))  # True
print(is_valid_parentheses("(]"))       # False
```

Complexity: Time O(n), Space O(n).

5.3 4.3 Problem: Next Greater Element

Statement: For each element, find the next element to its right that is greater than it (or -1 if none).

Approach: Use a **monotonic decreasing stack** of indices. Iterate through the array; while the current element is greater than the element at the index on top of the stack, that's the "next greater element" for that index — pop and record it.

```
def next_greater_element(nums):
    result = [-1] * len(nums)
    stack = []  # stores indices
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
```

```
            idx = stack.pop()
            result[idx] = num
        stack.append(i)
    return result

print(next_greater_element([2, 1, 2, 4, 3]))  # [4, 2, 4, -1, -1]
```

Complexity: Time O(n) — each element pushed/popped once, Space O(n).

5.4 4.4 Problem: Implement a Queue Using Two Stacks

Approach: Use `in_stack` for enqueue (push directly). For dequeue, if `out_stack` is empty, pour all elements from `in_stack` into `out_stack` (reversing order), then pop from `out_stack`.

```
class QueueUsingStacks:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def enqueue(self, x):
        self.in_stack.append(x)

    def dequeue(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack.pop() if self.out_stack else None
```

Complexity: Amortized O(1) per operation.

5.5 4.5 Problem: Sliding Window Maximum

Statement: Given an array and window size k, find the maximum in every window of size k.

Approach: Use a **monotonic decreasing deque** storing indices. Remove indices from the back if their value is smaller than the current element (they can never be the max again). Remove from the front if the index is out of the current window. The front of the deque is always the max.

```
from collections import deque

def sliding_window_max(nums, k):
    dq = deque()  # stores indices, values decreasing
    result = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

print(sliding_window_max([1,3,-1,-3,5,3,6,7], 3))  # [3,3,5,5,6,7]
```

Complexity: Time O(n), Space O(k).

5.6 4.6 Practice List

- Min Stack (get minimum in O(1))
- Evaluate Reverse Polish Notation
- Daily Temperatures (variant of next greater element)
- Largest Rectangle in Histogram
- Implement a Circular Queue

6 5. Trees

6.1 5.1 Concept

A tree is a hierarchical structure of nodes with a single root and no cycles. Binary trees (each node has ≤2 children) are the interview staple. Master the three DFS traversal orders and BFS (level order) — nearly every tree problem is a variation of one of these.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

6.2 5.2 Traversals

```
# Inorder (Left, Root, Right) — gives sorted order for a BST
def inorder(root, result=None):
    if result is None:
        result = []
    if root:
        inorder(root.left, result)
        result.append(root.val)
        inorder(root.right, result)
    return result

# Preorder (Root, Left, Right) — used to serialize a tree
def preorder(root, result=None):
    if result is None:
        result = []
    if root:
        result.append(root.val)
        preorder(root.left, result)
        preorder(root.right, result)
    return result

# Postorder (Left, Right, Root) — used to safely delete a tree, or evaluate expression tre
def postorder(root, result=None):
    if result is None:
        result = []
    if root:
        postorder(root.left, result)
        postorder(root.right, result)
        result.append(root.val)
    return result

# Level order (BFS) — uses a queue
from collections import deque
def level_order(root):
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
```

```
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

6.3 5.3 Problem: Maximum Depth of Binary Tree

Approach: Recursive: depth of a node = 1 + max(depth of left subtree, depth of right subtree). Base case: None node has depth 0.

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

Complexity: Time O(n), Space O(h) where h = tree height (recursion stack).

6.4 5.4 Problem: Validate a Binary Search Tree

Approach: A naive inorder-traversal-then-check-sorted works, but the cleaner approach is recursive range-checking: pass down a valid `(low, high)` range; each node's value must fall strictly within it, and the range narrows as you recurse left/right.

```
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.val < high):
        return False
    return (is_valid_bst(root.left, low, root.val) and
            is_valid_bst(root.right, root.val, high))
```

Complexity: Time O(n), Space O(h).

6.5 5.5 Problem: Lowest Common Ancestor (LCA) in a Binary Tree

Approach: Recursively search left and right subtrees. If both subtrees return a non-null result, the current node is the LCA. If only one side returns non-null, propagate that result upward.

```
def lowest_common_ancestor(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right:
        return root
    return left or right
```

Complexity: Time O(n), Space O(h).

6.6 5.6 Problem: Diameter of a Binary Tree

Statement: Find the length of the longest path between any two nodes (may or may not pass through the root).

Approach: For each node, the longest path through it = (height of left subtree) + (height of right subtree). Compute this while also computing height, updating a global/nonlocal max as you go — a classic "compute-and-update-simultaneously" recursion pattern.

```
def diameter_of_binary_tree(root):
    diameter = 0

    def height(node):
        nonlocal diameter
        if not node:
            return 0
        left_h = height(node.left)
        right_h = height(node.right)
        diameter = max(diameter, left_h + right_h)
        return 1 + max(left_h, right_h)

    height(root)
    return diameter
```

Complexity: Time O(n), Space O(h).

6.7 5.7 Problem: Serialize and Deserialize a Binary Tree

Approach: Use preorder traversal, marking None children explicitly with a sentinel (e.g., "#"). Deserialize by reconstructing recursively, consuming tokens in the same preorder sequence.

```
def serialize(root):
    result = []
    def helper(node):
        if not node:
            result.append("#")
            return
        result.append(str(node.val))
        helper(node.left)
        helper(node.right)
    helper(root)
    return ",".join(result)

def deserialize(data):
    values = iter(data.split(","))
    def helper():
        val = next(values)
        if val == "#":
            return None
        node = TreeNode(int(val))
        node.left = helper()
        node.right = helper()
        return node
    return helper()
```

Complexity: Time O(n), Space O(n).

6.8 5.8 Practice List

- Balanced Binary Tree Check
- Path Sum (root-to-leaf, target sum)
- Kth Smallest Element in a BST
- Convert Sorted Array to Balanced BST
- Right Side View of a Binary Tree
- Trie (Prefix Tree) implementation — used for autocomplete/word search problems

7 6. Graphs

7.1 6.1 Concept

A graph is a set of nodes (vertices) connected by edges — can be directed/undirected, weighted/unweighted. Represented as an **adjacency list** (space-efficient, `dict[node] = [neighbors]`) or **adjacency matrix** (fast edge lookup, $O(V^2)$ space). Master DFS, BFS, and know when to reach for Dijkstra, Union-Find, or Topological Sort.

```
# Adjacency list representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
```

7.2 6.2 Problem: Depth-First Search (DFS) and Breadth-First Search (BFS)

Approach: DFS explores as deep as possible before backtracking (uses a stack or recursion). BFS explores level by level (uses a queue) — the go-to for shortest path in an **unweighted** graph.

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited

from collections import deque
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

Complexity: Both Time O(V + E), Space O(V).

7.3 6.3 Problem: Detect Cycle in a Graph

Approach (Undirected): DFS while tracking the parent node — if you visit a node that's already visited and it's NOT the parent, there's a cycle.

Approach (Directed): DFS while tracking nodes in the *current recursion path* (a "gray" set). If you reach a node that's already in the current path, there's a cycle (this is different from just "visited," since a node could be fully processed and revisited safely in a DAG).

```python
# Directed graph cycle detection
def has_cycle_directed(graph):
    visited = set()
    in_path = set()

    def dfs(node):
        visited.add(node)
        in_path.add(node)
        for neighbor in graph.get(node, []):
            if neighbor in in_path:
                return True
            if neighbor not in visited and dfs(neighbor):
                return True
        in_path.remove(node)
        return False

    for node in graph:
        if node not in visited:
            if dfs(node):
                return True
    return False
```

Complexity: Time O(V + E), Space O(V).

7.4 6.4 Problem: Topological Sort

Statement: Order nodes of a DAG such that for every directed edge u→v, u comes before v.

Approach (Kahn's Algorithm / BFS-based): Compute in-degree of every node. Start with all nodes having in-degree 0, process them (add to result), and decrement in-degree of their neighbors — add neighbors to the queue when their in-degree hits 0.

```python
from collections import deque

def topological_sort(graph, num_nodes):
    in_degree = {i: 0 for i in range(num_nodes)}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    queue = deque([n for n in in_degree if in_degree[n] == 0])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph.get(node, []):
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return result if len(result) == num_nodes else []  # empty if cycle exists
```

Complexity: Time O(V + E), Space O(V).

7.5 6.5 Problem: Number of Islands

Statement: Given a 2D grid of '1's (land) and '0's (water), count the number of islands (connected groups of '1's, horizontally/vertically connected).

Approach: For each unvisited land cell, run DFS/BFS to mark the entire connected component as visited, and increment the island count once per component.

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r, c):
        if (r < 0 or r >= rows or c < 0 or c >= cols or
                grid[r][c] == '0' or (r, c) in visited):
            return
        visited.add((r, c))
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                dfs(r, c)
                count += 1
    return count
```

Complexity: Time O(rows × cols), Space O(rows × cols) worst case (recursion stack + visited set).

7.6 6.6 Problem: Dijkstra's Algorithm (Shortest Path, Weighted, Non-negative)

Approach: Use a min-heap (priority queue) keyed by current shortest distance. Greedily pop the closest unvisited node, relax (update) distances to its neighbors, and push updated distances.

```
import heapq

def dijkstra(graph, start):
    # graph: {node: [(neighbor, weight), ...]}
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    heap = [(0, start)]

    while heap:
        curr_dist, node = heapq.heappop(heap)
        if curr_dist > distances[node]:
            continue
        for neighbor, weight in graph[node]:
            distance = curr_dist + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
```

```
                heapq.heappush(heap, (distance, neighbor))
    return distances
```

Complexity: Time O((V + E) log V) with a binary heap, Space O(V).

7.7 6.7 Problem: Union-Find (Disjoint Set Union) — for Connected Components / Cycle Detection

Approach: Maintain a `parent` array. `find(x)` follows parent pointers to the root (with **path compression** for speed). `union(x, y)` merges two sets by attaching one root to the other (with **union by rank** for balance). Widely used for Kruskal's MST and detecting cycles in undirected graphs efficiently.

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)
        if root_x == root_y:
            return False  # already connected -> would form a cycle
        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1
        return True
```

Complexity: Nearly O(1) amortized per operation (inverse Ackermann function).

7.8 6.8 Practice List

- Clone a Graph
- Course Schedule (cycle detection application)
- Word Ladder (BFS shortest transformation)
- Minimum Spanning Tree (Kruskal's / Prim's)
- Bellman-Ford Algorithm (handles negative weights)
- Bipartite Graph Check

8 7. Dynamic Programming (DP)

8.1 7.1 Concept

DP solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation. Two approaches: - **Top-down (Memoization)**: recursion + a cache (dict/array). - **Bottom-up (Tabulation)**: build a table iteratively from base cases upward.

How to recognize a DP problem: the problem asks for an optimal value (min/max/count of ways), and it has **optimal substructure** (the answer can be built from answers to smaller subproblems) and **overlapping subproblems** (the same subproblem recurs multiple times in the recursion tree).

General approach for any DP problem: 1. Define the state — what does `dp[i]` (or `dp[i][j]`) represent? 2. Find the recurrence relation — how does `dp[i]` relate to smaller states? 3. Identify base cases. 4. Decide iteration order (bottom-up) or add memoization (top-down). 5. Optimize space if only the last few states are needed (rolling array).

8.2 7.2 Problem: Fibonacci Number (the "hello world" of DP)

Approach: `dp[i] = dp[i-1] + dp[i-2]`. Demonstrates memoization vs tabulation vs O(1) space.

```
# Top-down (memoization)
def fib_memo(n, memo={}):
    if n <= 1:
        return n
    if n not in memo:
        memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# Bottom-up (tabulation), O(1) space
def fib_tab(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

Complexity: Time O(n), Space O(n) memo / O(1) tabulation optimized.

8.3 7.3 Problem: 0/1 Knapsack

Statement: Given weights and values of n items and a capacity W, maximize value without exceeding W, using each item at most once.

Approach: State: `dp[i][w]` = max value using first i items with capacity w. Recurrence: for each item, either skip it (`dp[i-1][w]`) or take it if it fits (`value[i] + dp[i-1][w-weight[i]]`) — take the max of both choices.

```
def knapsack_01(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
```

```
        for w in range(capacity + 1):
            dp[i][w] = dp[i-1][w]  # don't take item i-1
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], values[i-1] + dp[i-1][w - weights[i-1]])
    return dp[n][capacity]

print(knapsack_01([1,3,4,5], [1,4,5,7], 7))  # 9
```

Complexity: Time O(n × W), Space O(n × W) — can be optimized to O(W) with a 1D rolling array.

8.4 7.4 Problem: Longest Common Subsequence (LCS)

Statement: Given two strings, find the length of their longest common subsequence (not necessarily contiguous).

Approach: State: `dp[i][j]` = LCS length of `s1[0:i]` and `s2[0:j]`. If characters match, `dp[i][j] = 1 + dp[i-1][j-1]`; else `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

print(lcs("abcde", "ace"))  # 3
```

Complexity: Time O(m×n), Space O(m×n).

8.5 7.5 Problem: Longest Increasing Subsequence (LIS)

Statement: Find the length of the longest strictly increasing subsequence in an array.

Approach (O(n²) DP): `dp[i]` = length of LIS ending at index i. For each i, check all j < i; if `nums[j] < nums[i]`, `dp[i] = max(dp[i], dp[j] + 1)`.

Faster O(n log n) approach: maintain a list `tails` where `tails[k]` = smallest possible tail value of an increasing subsequence of length k+1; use binary search to find where each new element fits.

```
# O(n^2) version — clearer for interviews to explain first
def lis_length(nums):
    if not nums:
        return 0
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
```

```
    return max(dp)

print(lis_length([10,9,2,5,3,7,101,18]))  # 4 ([2,3,7,101] or [2,3,7,18])

# O(n log n) version using binary search
import bisect
def lis_length_fast(nums):
    tails = []
    for num in nums:
        idx = bisect.bisect_left(tails, num)
        if idx == len(tails):
            tails.append(num)
        else:
            tails[idx] = num
    return len(tails)
```

Complexity: $O(n^2)$ or O(n log n) with binary search.

8.6 7.6 Problem: Coin Change (Minimum Coins)

Statement: Given coin denominations and a target amount, find the minimum number of coins to make that amount (or -1 if impossible).

Approach: State: `dp[amt]` = minimum coins to make `amt`. Base case: `dp[0] = 0`. For each amount from 1 to target, try every coin: `dp[amt] = min(dp[amt], dp[amt - coin] + 1)` if `amt - coin >= 0`.

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for amt in range(1, amount + 1):
        for coin in coins:
            if coin <= amt:
                dp[amt] = min(dp[amt], dp[amt - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

print(coin_change([1, 2, 5], 11))  # 3 (5+5+1)
```

Complexity: Time O(amount × num_coins), Space O(amount).

8.7 7.7 Problem: Edit Distance (Levenshtein Distance)

Statement: Find the minimum number of insertions, deletions, or substitutions to convert one string into another.

Approach: State: `dp[i][j]` = edit distance between `s1[0:i]` and `s2[0:j]`. If characters match, no operation needed: `dp[i][j] = dp[i-1][j-1]`. Otherwise, take 1 + min of (insert, delete, replace) from the three neighboring states.

```
def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
```

```
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],    # delete
                                   dp[i][j-1],    # insert
                                   dp[i-1][j-1])  # replace
    return dp[m][n]

print(edit_distance("horse", "ros"))  # 3
```

Complexity: Time O(m×n), Space O(m×n).

8.8 7.8 Problem: House Robber

Statement: Given houses with money, rob houses to maximize total loot, without robbing two adjacent houses.

Approach: State: `dp[i]` = max loot considering first i houses. Recurrence: `dp[i] = max(dp[i-1], dp[i-2] + nums[i])` — either skip house i (take best up to i-1) or rob it (best up to i-2 plus current).

```
def rob(nums):
    prev2, prev1 = 0, 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + num)
    return prev1

print(rob([2,7,9,3,1]))  # 12
```

Complexity: Time O(n), Space O(1).

8.9 7.9 Practice List

- Unique Paths (grid DP)
- Partition Equal Subset Sum
- Word Break
- Maximum Product Subarray
- Palindrome Partitioning (min cuts)
- Matrix Chain Multiplication

9 8. Greedy Algorithms

9.1 8.1 Concept

Greedy algorithms make the locally optimal choice at each step, hoping it leads to a global optimum. Works only when the problem has the **greedy-choice property** — unlike DP, there's no "trying all options and picking the best," so greedy is faster but requires proving (or trusting) that local optimality leads to global optimality.

9.2 8.2 Problem: Activity Selection / Non-overlapping Intervals

Statement: Given intervals, find the maximum number of non-overlapping intervals you can select.

Approach: Sort intervals by **end time**. Greedily pick the interval with the earliest end time, then skip all intervals that overlap with it, and repeat. Sorting by end time (not start time) is the key insight — it always leaves the most room for future intervals.

```python
def max_non_overlapping_intervals(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end time
    count = 0
    last_end = float('-inf')
    for start, end in intervals:
        if start >= last_end:
            count += 1
            last_end = end
    return count

print(max_non_overlapping_intervals([[1,3],[2,4],[3,5],[7,9]]))  # 3
```

Complexity: Time O(n log n) (sorting dominates), Space O(1).

9.3 8.3 Problem: Fractional Knapsack

Statement: Like 0/1 knapsack, but items can be broken into fractions.

Approach: Sort items by value/weight ratio in descending order. Greedily take as much as possible of the highest-ratio item first, then move to the next, until the capacity is filled. (Unlike 0/1 knapsack, greedy works here because fractional items make the greedy-choice property hold.)

```python
def fractional_knapsack(weights, values, capacity):
    items = sorted(zip(values, weights), key=lambda x: x[0]/x[1], reverse=True)
    total_value = 0.0
    for value, weight in items:
        if capacity >= weight:
            capacity -= weight
            total_value += value
        else:
            total_value += value * (capacity / weight)
            break
    return total_value

print(fractional_knapsack([10, 20, 30], [60, 100, 120], 50))  # 240.0
```

Complexity: Time O(n log n), Space O(1).

9.4 8.4 Problem: Jump Game (Minimum Jumps / Reachability)

Statement: Given an array where `nums[i]` = max jump length from position i, determine if you can reach the last index (or the minimum number of jumps to do so).

Approach: Track the farthest index reachable so far while iterating. If at any point the current index exceeds the farthest reachable index, you're stuck (return False). For minimum jumps, track the "current jump's boundary" and increment a jump counter whenever you must extend past it.

```python
def can_jump(nums):
    farthest = 0
    for i, num in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + num)
    return True

print(can_jump([2,3,1,1,4]))  # True
print(can_jump([3,2,1,0,4]))  # False
```

Complexity: Time O(n), Space O(1).

9.5 8.5 Practice List

- Gas Station (circular tour feasibility)
- Job Sequencing with Deadlines
- Huffman Coding
- Minimum Number of Platforms (interval scheduling variant)

10 Complexity Cheat Sheet

Pattern	Typical Time Complexity	When to Use
Two pointers	O(n)	Sorted arrays, pair/triplet sum problems
Sliding window	O(n)	Substring/subarray with a constraint
Hash map lookup	O(n)	Existence/frequency checks, complements
Binary search	O(log n)	Sorted data, “search space” problems
DFS/BFS	O(V+E)	Traversal, connectivity, shortest path (unweighted)
Dijkstra	O((V+E) log V)	Shortest path, weighted, non-negative
DP (2D table)	O(n×m)	Optimal value with 2 changing parameters
Greedy + sort	O(n log n)	Interval scheduling, ratio-based selection

End of DSA Guide. Practice each pattern until you recognize it within the first 30 seconds of reading a new problem — that pattern recognition, not memorized code, is what interviews actually test.